Programmation orientée objet en Python

Classes, attributs, méthodes — Encapsulation et héritage

BTS CIEL — Travaux pratiques

Nom, prénom : .....

Classe : .....

Date : .....

Objectifs du TP

- Lire une classe Python et identifier son constructeur, ses attributs et ses méthodes.
- Instancier un objet et appeler ses méthodes.
- Compléter une classe existante en ajoutant une méthode.
- Protéger les données d'un objet par l'encapsulation (attributs privés, accesseurs, mutateurs).
- Écrire un mutateur qui valide la donnée avant de l'enregistrer.
- Construire une classe fille par héritage, redéfinir une méthode et appeler le constructeur parent.

Consignes : chaque question se traite d'abord sur le papier, puis se vérifie dans l'interpréteur Python (fichier `tp_poo.py`). Le code rendu doit s'exécuter sans erreur.

Vocabulaire de référence : Une **classe** est un modèle. Un **objet** (ou **instance**) est une réalisation concrète de ce modèle. Un **attribut** est une donnée portée par l'objet. Une **méthode** est une fonction définie dans la classe et appelée sur un objet. Le paramètre `self` désigne l'objet sur lequel la méthode est appelée.

Partie A — Classes, attributs et méthodes

Exercice 1 — La classe `Voiture`. Lire le code suivant, puis répondre aux questions.

```
class Voiture:
    def __init__(self, marque, couleur):
        self.marque = marque
        self.couleur = couleur

    def afficher_details(self):
        return f"Voiture {self.marque} de couleur {self.couleur}"
```

Questions

1. Que fait la méthode `__init__` ?

   .....

   .....

2. Combien d'attributs possède la classe `Voiture` ?

   .....

   .....

3. Que retourne la méthode `afficher_details` ?

   .....

. . . . . .

À compléter : créer une instance de `Voiture` de marque `"Toyota"` et de couleur `"Bleu"`, appeler `afficher_details` et afficher le résultat.

Exercice 2 — Compléter la classe `Chien`.

```
class Chien:
    def __init__(self, nom, age):
        self.nom = nom
        self.age = age

    # Méthode aboyer qui retourne "Wouf !"
```

Questions

1. Quels sont les attributs de la classe `Chien` ?

   . . . . . .

   . . . . . .

2. Comment appelle-t-on la méthode `__init__` dans le contexte de la POO ?

   . . . . . .

   . . . . . .

3. Ajouter une méthode `aboyer` qui retourne `"Wouf !"`.
4. Créer un objet `Chien` nommé `"Rex"`, âgé de 3 ans, et le faire aboyer.

Exercice 3 — La classe `Livre`.

```
class Livre:
    def __init__(self, titre, auteur):
        self.titre = titre
        self.auteur = auteur

    def description(self):
        return f"{self.titre} par {self.auteur}"
```

Questions

1. Que fait la méthode `description` ?

   .....

   .....

2. Combien de paramètres la méthode `__init__` attend-elle, en incluant `self` ?

   .....

   .....

3. Créer un objet `Livre` de titre `"Le Petit Prince"` et d'auteur `"Antoine de Saint-Exupéry"`, puis afficher sa description.

Exercice 4 — La classe `Rectangle`.

```
class Rectangle:
    def __init__(self, longueur, largeur):
        self.longueur = longueur
        self.largeur = largeur

    # Méthode calculer_aire qui retourne l'aire du rectangle
```

Questions

1. Quels sont les attributs de la classe `Rectangle` ?

   .....

   .....

2. Quelle formule mathématique faut-il utiliser pour calculer l'aire ?

   .....

   .....

3. Ajouter une méthode `calculer_aire` qui retourne l'aire du rectangle.
4. Créer un rectangle de longueur 5 et de largeur 3, puis afficher son aire.

Exercice 5 — Création complète d'une classe `Etudiant`. On souhaite modéliser un étudiant par son nom et sa note.

1. Créer une classe `Etudiant` possédant un constructeur `__init__` prenant en paramètres `nom` et `note`, et deux attributs d'instance `self.nom` et `self.note`.
2. Ajouter une méthode `a_reussi` qui retourne `True` si la note est supérieure ou égale à 10, et `False` sinon.
3. Tester la classe en créant un étudiant avec une note de 8.
4. Ajouter une méthode `augmenter_note` qui augmente la note de 2 points.
5. Créer un étudiant nommé `"Alice"` avec une note de 15 et afficher si elle a réussi, grâce à la méthode `a_reussi`.

Partie B — Encapsulation et héritage

Encapsulation : Préfixer un attribut par deux soulignés (`self.__nom`) déclenche le *name mangling* : Python renomme l'attribut en `_Classe__nom`, ce qui le rend inaccessible par son nom d'origine depuis l'extérieur. On y accède alors par un **accesseur** (*getter*, `get_nom`) et on le modifie par un **mutateur** (*setter*, `set_nom`), ce qui permet de contrôler les valeurs acceptées.

Héritage : `class Chat(Animal)` signifie que `Chat` **hérite** de `Animal` : la classe fille reçoit les attributs et méthodes de la classe mère. Elle peut en ajouter, ou **redéfinir** une méthode existante (*override*). `super().__init__(...)` appelle le constructeur de la classe mère.

Exercice 1 — Encapsulation simple avec attribut privé.

```
class Personne:
    def __init__(self, nom, age):
        self.__nom = nom
        self.__age = age

    def get_nom(self):
        return self.__nom

    # Méthode get_age()
```

Questions

1. Pourquoi les attributs `__nom` et `__age` sont-ils privés ?

   .....

   .....

2. Comment accède-t-on à l'attribut `__nom` ?

   .....

   .....

3. Ajouter une méthode `get_age` pour accéder à l'attribut `__age`.
4. Créer une instance de `Personne` de nom `"Luc"` et d'âge 25, puis afficher son nom.

Exercice 2 — Ajouter un mutateur.

```
class CompteBancaire:
    def __init__(self, titulaire, solde):
        self.__titulaire = titulaire
        self.__solde = solde

    def get_solde(self):
        return self.__solde
```

Questions

1. Que signifie l'encapsulation dans cette classe ?

   .....

   .....

2. Que fait la méthode `get_solde` ?

   .....

   .....

3. Créer un compte bancaire pour `"Emma"` avec un solde de 500 et afficher le solde.
4. Ajouter une méthode `set_solde` pour modifier le solde.

Exercice 3 — Encapsulation avec validation.

```
class Etudiant:
    def __init__(self, nom, note):
        self.__nom = nom
        self.__note = note

    def get_note(self):
        return self.__note

    # Méthode set_note qui accepte uniquement des notes entre 0 et 20
```

Questions

1. Pourquoi utiliser un mutateur pour la note ?

   .....

   .....

2. Que se passe-t-il si on essaie d'accéder directement à `__note` ?

   .....

. . . . . .

3. Ajouter une méthode `set_note` qui n'accepte que des notes entre 0 et 20.
4. Créer un étudiant nommé `"Paul"` avec une note de 15, puis essayer de changer la note à 25.

Exercice 4 — Héritage simple.

```
class Animal:
    def __init__(self, nom):
        self.__nom = nom

    def get_nom(self):
        return self.__nom

    def faire_son(self):
        return "Son generique"

class Chat(Animal):
    # Méthode faire_son pour retourner "Miaou"
```

Questions

1. Quelle classe est la classe parent ?

   . . . . . .

   . . . . . .

2. Que fait l'héritage dans ce cas ?

   . . . . . .

   . . . . . .

3. Redéfinir la méthode `faire_son` pour qu'elle retourne `"Miaou"`.
4. Créer un `Chat` nommé `"Minou"` et afficher son son.

Exercice 5 — Héritage avec nouvel attribut.

```
class Vehicule:
    def __init__(self, marque):
        self.__marque = marque

    def get_marque(self):
        return self.__marque

class Voiture(Vehicule):
    def __init__(self, marque, couleur):
        super().__init__(marque)
        self.__couleur = couleur

    # À compléter : ajoutez une méthode get_couleur
```

Questions

1. Que fait `super().__init__(marque)` ?

   .....

   .....

2. Combien d'attributs privés possède la classe `Voiture` ?

   .....

   .....

3. À compléter : ajouter une méthode `get_couleur`.

Exercice 6 — Encapsulation et mutateur avec validation.

```
class Livre:
    def __init__(self, titre, prix):
        self.__titre = titre
        self.__prix = prix

    def get_prix(self):
        return self.__prix

    # À compléter : ajoutez une méthode set_prix qui refuse les prix négatifs
```

Questions

1. Pourquoi le prix est-il un attribut privé ?

   .....

   .....

2. Que fait un mutateur avec validation ?

   . . . . . .

   . . . . . .

3. Créer un `Livre` intitulé `"Python 101"` avec un prix de 30, puis essayer de définir un prix de -10.

Synthèse

Notion	Ce qu'il faut retenir
Classe	Modèle décrivant des données et des comportements : `class Voiture:`
Objet / instance	Réalisation concrète de la classe : `v = Voiture("Toyota", "Bleu")`
Constructeur	`__init__`, appelé automatiquement, initialise les attributs
`self`	Référence à l'objet courant ; premier paramètre de toute méthode d'instance
Attribut d'instance	Donnée propre à l'objet : `self.marque`
Méthode	Fonction définie dans la classe, appelée par `objet.methode()`
Attribut privé	`self.__prix` : inaccessible depuis l'extérieur (*name mangling*)
Accesseur (*getter*)	`get_prix()` : lit un attribut privé
Mutateur (*setter*)	`set_prix(v)` : écrit un attribut privé après validation
Héritage	`class Voiture(Vehicule)` : la fille reçoit les membres de la mère
`super()`	Accède à la classe mère, notamment `super().__init__(...)`
Redéfinition	La fille réécrit une méthode de la mère (`faire_son`)

Erreurs les plus fréquentes. oublier `self` dans la définition d'une méthode ; confondre `return` (retourne une valeur) et `print` (affiche) ; écrire `objet.__attribut` depuis l'extérieur ; oublier `super().__init__(...)` dans le constructeur d'une classe fille ; appeler une méthode sans les parenthèses (`v.afficher_details` retourne la méthode, pas son résultat).